

CS231n 4-6

김동원

목차

Backpropagation

Fully connected Layer

Convolutional Layer

Data Preprocessing

Weight Initialization

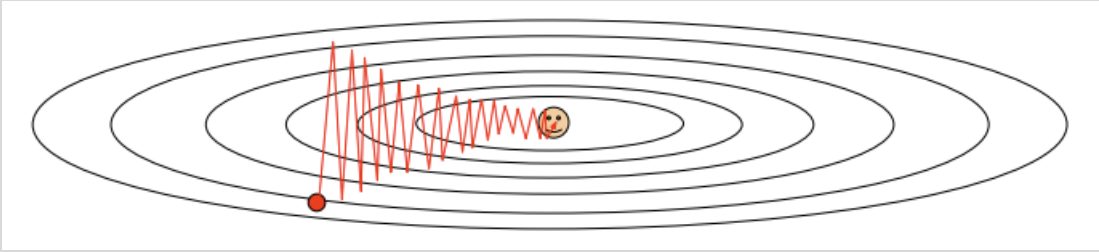
Batch Normalization

Hyperparameter Optimization

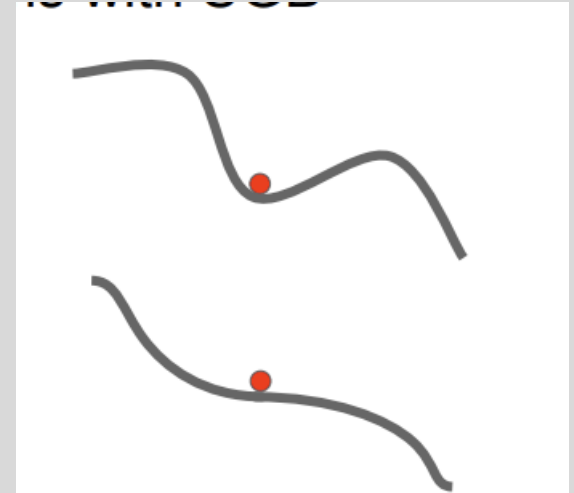
Optimization

SGD

미니배치를 사용하여 가중치를 업데이트 하는 방법



한 쪽 방향으로 loss가 크게 변하는 경우 SGD는 진동하는 것 처럼 업데이트 될 것이고 실제 입력데이터에서 학습에 문제가 된다



지역최적해와 saddle point를 벗어나지 못한다.

Optimization

SGD+Momentum

기울기 + 방향의 속도 까지 고려하는 방법

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x += learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

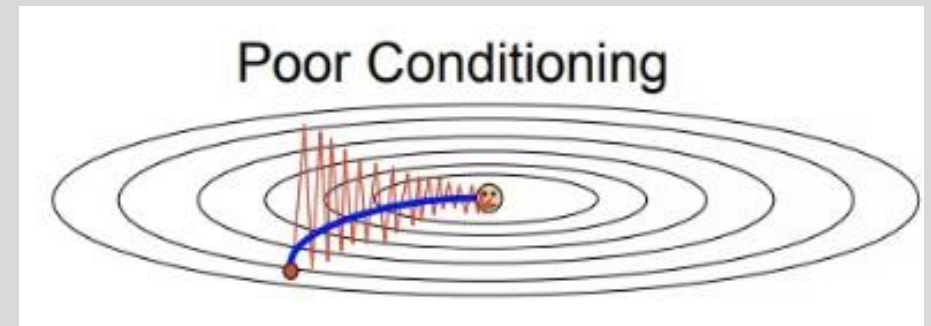
```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x += learning_rate * vx
```

- Build up “velocity” as a running mean of gradients
- Rho gives “friction”; typically rho=0.9 or 0.99

속도(velocity)는 가중합을 의미하고

$$v_{t+1} = \rho v_t + \nabla f(x)$$

ρ 는 보통 0.9, 0.99를 가진다

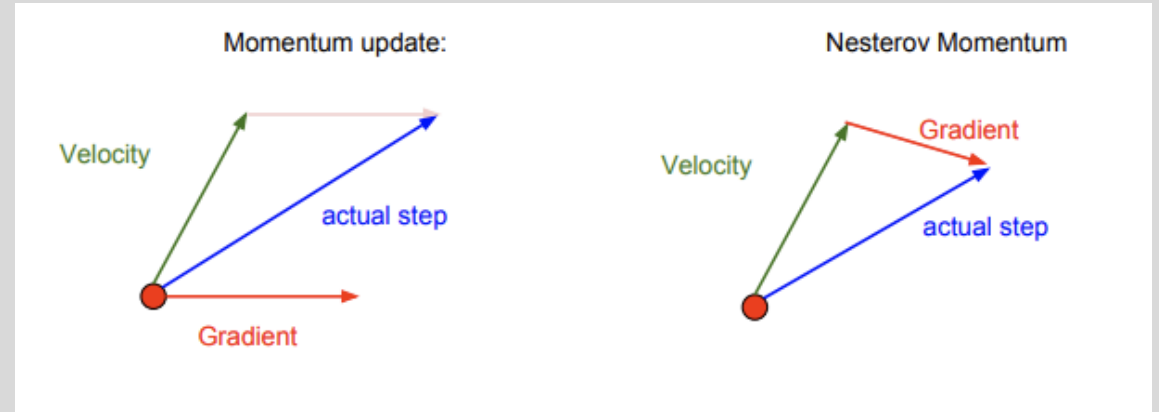


진동이 줄어든다

Optimization

Nesterov Momentum

이동 방향을 예상하여 기울기를 계산한다.



$$\begin{aligned} v_{t+1} &= \rho v_t - \alpha \nabla f(x_t + \rho v_t) \\ x_{t+1} &= x_t + v_{t+1} \end{aligned}$$

Annoying, usually we want update in terms of $x_t, \nabla f(x_t)$

Change of variables $\tilde{x}_t = x_t + \rho v_t$ and rearrange:

$$\begin{aligned} v_{t+1} &= \rho v_t - \alpha \nabla f(\tilde{x}_t) \\ \tilde{x}_{t+1} &= \tilde{x}_t - \rho v_t + (1 + \rho)v_{t+1} \\ &= \tilde{x}_t + v_{t+1} + \rho(v_{t+1} - v_t) \end{aligned}$$

```
dx = compute_gradient(x)
old_v = v
v = rho * v - learning_rate * dx
x += -rho * old_v + (1 + rho) * v
```

기본적인 모멘텀 최적화보다 훈련이 빠르다

Optimization

AdaGrad

Velocity 대신에 기울기의 제곱을 사용하여 기울기를 구한다.
기울기의 제곱합의 제곱근으로 나눠서 학습한다

AdaGrad

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared += dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

제곱합이 계속 커지면서 학습이 빠르게 멈추게 된다.

Optimization

RMSProp

AdaGrad가 너무 빨리 학습이 멈추는 것을 해결하기 위해 기울기 제곱합에 decay를 곱하여 값이 크게 늘어나지 않게 한다.

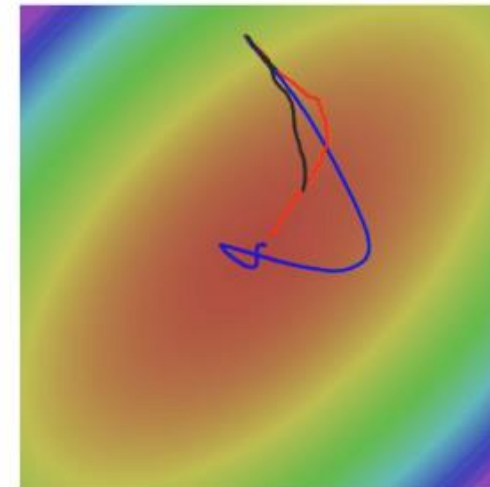
AdaGrad

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared += dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



- SGD
- SGD+Momentum
- RMSProp

Optimization

Adam

모멘텀과 RMSProp의 아이디어를 합친것

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

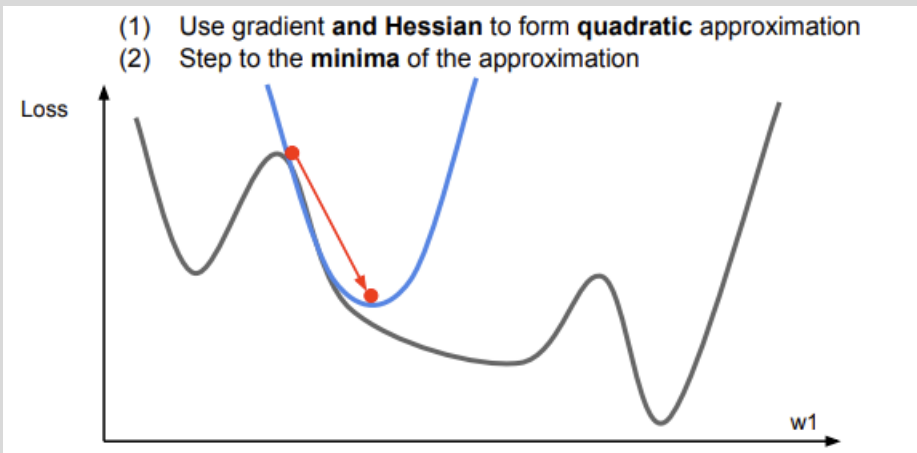
모멘텀과 RMSProp의 방법을 합쳐서 학습률을 정하는데
first_unbias와 second_unbias를 통해 초반의 step을 정하는데
도움을 준다

Learning rate

Learning rate decay

SGD모멘텀의 경우에 일반적으로 사용하고, adam의 경우에는 그렇지 않다. Learning rate decay는 옵티마이저와 일차적인 파라미터 성능 확인에 혼란을 줄 수 있다.

Second-Order Optimization



Hessian 매트릭스를 이용하여 기울기를 2차식으로 근사할 수 있고 바로 최적해를 찾을 수 있게 되므로 learning rate를 사용하지 않아도 된다

Hessian 매트릭스의 크기가 매우 크고 역행렬을 구해야 하기 때문에 계산이 어렵다

Ensemble & Regularization

Ensemble

처음 입력된 데이터에서의 성능을 높이기 위해 독립적으로 학습한 여러 모델을 이용하여 예측한다. 여러 모델이 될 수 있고, epoch 마다 학습된 모델, 파라미터들 간의 앙상블도 가능하다

Regularization

단일 모델로 좋은 성능을 내게 해준다. L1, l2 등이 있고 Dropout, Data Augment, Fractional Max Polling,

Dropout

학습 시 뉴런을 확률적으로 비활성화 한다
테스트시에는 뉴런의 랜덤한 활성화 대신 연산한 값에서 dropout 확률을 곱한다.

CPU와 GPU

CPU

연산이 매우 빠르지만 메모리를 가져오는 구간에서 병목이 발생

GPU

GPU는 자신만의 메모리를 가지고 있어 CPU의 병목을 줄일 수 있음 또한 딥러닝에서 많이 사용하는 벡터의 곱이 GPU의 연산처리 방법과 잘 맞는다

CUDA를 통해 GPU에서 실행되는 코드를 직접 짤 수 있다. 하지만 GPU의 자원할당이 매우 어려워 라이브러리를 이용한다. cuBLAS, cuFFT, cuDNN

Fully connected layer 구현

```
indim = input_dim
for i in range(len(hidden_dims)):
    self.params['W'+str(i+1)] = np.random.randn(indim,hidden_dims[i])*weight_scale
    self.params['b'+str(i+1)] = np.zeros(hidden_dims[i],)
    if normalization == 'batchnorm':
        self.params['gamma'+str(i+1)] = np.ones_like(self.params['b'+str(i+1)])
        self.params['beta'+str(i+1)] = np.zeros_like(self.params['b'+str(i+1)])
    indim = hidden_dims[i]

if i+2 == self.num_layers:
    self.params['W'+str(i+2)] = np.random.randn(indim,num_classes)*weight_scale
    self.params['b'+str(i+2)] = np.zeros(num_classes,)
```

Weight와 bias 생성, batchnorm이 적용되면 gamma와 beta도 생성하고 마지막 layer는 class의 개수만큼

Score계산 – score와 파라미터를 저장

```
save = [0]
if self.use_dropout:
    dropsave=[0]

for l in range(1, self.num_layers+1):
    if l ==1:
        if self.normalization == 'batchnorm':
            out, cache = affine_batchnorm_relu_forward(X,self.params['W'+str(l)], self.params['b'+str(l)],
                self.params['gamma'+str(l)],self.params['beta'+str(l)],self.bn_params[l-1])
            save.append(cache)
        else:
            out, cache = affine_relu_forward(X,self.params['W'+str(l)], self.params['b'+str(l)])
            save.append(cache)
        if self.use_dropout:
            out, cache = dropout_forward(out, self.dropout_param)
            dropsave.append(cache)
        continue

    if l == self.num_layers:
        scores, cache = affine_forward(out,self.params['W'+str(l)],self.params['b'+str(l)])
        save.append(cache)
        continue

    if self.normalization == 'batchnorm':
        out, cache = affine_batchnorm_relu_forward(out, self.params[f'W{l}'], self.params[f'b{l}'],
            self.params[f'gamma{l}'],self.params[f'beta{l}'],self.bn_params[l-1])
        save.append(cache)
    else:
        out, cache = affine_relu_forward(out,self.params[f'W{l}'],self.params[f'b{l}'])
        save.append(cache)

    if self.use_dropout:
        out, cache = dropout_forward(out, self.dropout_param)
        dropsave.append(cache)
    continue
```

Fully connected layer 구현

Loss 계산 - loss와 기울기를 반환

```
loss, dx = softmax_loss(scores, y)

for l in range(self.num_layers, 0, -1):
    if l == self.num_layers:
        dx, dw, db = affine_backward(dx, save[l])
        dw += self.reg * self.params['W'+str(l)]
        grads[f'W{l}'] = dw
        grads[f'b{l}'] = db
        loss += np.sum(0.5*self.reg*self.params[f'W{l}']**2)
        continue

    if self.use_dropout:
        dx = dropout_backward(dx, dropsave[l])

    if self.normalization == 'batchnorm':
        dx, dw, db, dgamma, dbeta = affine_batchnorm_relu_backward(dx, save[l])
        dw += self.reg * self.params[f'W{l}']
        grads[f'W{l}'] = dw
        grads[f'b{l}'] = db
        grads[f'gamma{l}'] = dgamma
        grads[f'beta{l}'] = dbeta
        loss += np.sum(0.5 * self.reg * self.params[f'W{l}']**2)

    else:
        dx, dw, db = affine_relu_backward(dx, save[l])
        dw += self.reg * self.params[f'W{l}']
        grads[f'W{l}'] = dw
        grads[f'b{l}'] = db
        loss += np.sum(0.5* self.reg * self.params[f'W{l}']**2)
```

Batchnorm 구현

Forward – train시

```
sample_mean = np.mean(x,axis=0)
sample_var = np.var(x,axis=0)

running_mean = momentum * running_mean + (1 - momentum) * np.mean(x,axis=0)
running_var = momentum * running_var + (1 - momentum) * sample_var

x_normal = (x - sample_mean) / np.sqrt(sample_var + eps)
out = gamma * x_normal + beta

cache = (x, sample_mean, sample_var, gamma)
```

Forward – test시

```
elif mode == "test":
    #####
    # TODO: Implement the test-time forward pass for batch normalization. #
    # Use the running mean and variance to normalize the incoming data,   #
    # then scale and shift the normalized data using gamma and beta.      #
    # Store the result in the out variable.                                #
    #####
    # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
    x_hat = (x - running_mean) / np.sqrt(running_var + eps)
    out = gamma * x_hat + beta
    pass
```

Batchnorm 구현

Backward시 파라미터 업데이트 방법

- **x_normal**: 입력 데이터를 정규화합니다.
$$x_{normal} = (x - \text{sample_mean}) / \sqrt{\text{sample_var} + \text{eps}}$$
- **dgamma**: gamma에 대한 gradient를 계산합니다.
$$d\gamma = \sum(dout \cdot x_{normal}, \text{axis}=0)$$
- **dbeta**: beta에 대한 gradient를 계산합니다.
$$dbeta = \sum(dout, \text{axis}=0)$$
- **dx_normal**: 정규화된 입력에 대한 gradient를 계산합니다.
$$dx_{normal} = dout \cdot \gamma$$
- **dlvar**: 분산에 대한 gradient를 계산합니다.
$$dlvar = \sum(dx_{normal} \cdot (x - \text{sample_mean}) \cdot -0.5 \cdot (\text{sample_var} + \text{eps})^{-1.5}, \text{axis}=0)$$
- **dlmean**: 평균에 대한 gradient를 계산합니다.
$$dlmean = \sum(dx_{normal} \cdot -1 / \sqrt{\text{sample_var} + \text{eps}}, \text{axis}=0) + dlvar \cdot \sum(-2 \cdot (x - \text{sample_mean}), \text{axis}=0) / N$$
- **dx**: 입력 데이터에 대한 gradient를 계산합니다.
$$dx = dx_{normal} \cdot 1 / \sqrt{\text{sample_var} + \text{eps}} + dlvar \cdot 2 \cdot (x - \text{sample_mean}) / N + dlmean / N$$

```
eps = 1e-5
N = dout.shape[0]
x, sample_mean, sample_var, gamma = cache
x_normal = (x - sample_mean) / np.sqrt(sample_var + eps)
dgamma = np.sum(dout * x_normal, axis = 0)
dbeta = np.sum(dout, axis = 0)
dx_normal = dout * gamma
dlvar = np.sum(dx_normal * (x - sample_mean) * -0.5 * (sample_var + eps)**-1.5, axis = 0)
dlmean = np.sum(dx_normal * -1 / np.sqrt(sample_var + eps), axis = 0)
+ dlvar * np.sum(-2 * (x - sample_mean), axis = 0) / N
dx = dx_normal * 1 / np.sqrt(sample_var + eps) + dlvar * 2 * (x - sample_mean) / N + dlmean / N
```

Batchnorm 구현

Backward시 파라미터 업데이트 방법

- **x_normal**: 입력 데이터를 정규화합니다.
$$x_{normal} = (x - \text{sample_mean}) / \sqrt{\text{sample_var} + \text{eps}}$$
- **dgamma**: gamma에 대한 gradient를 계산합니다.
$$d\gamma = \sum(dout \cdot x_{normal}, \text{axis}=0)$$
- **dbeta**: beta에 대한 gradient를 계산합니다.
$$dbeta = \sum(dout, \text{axis}=0)$$
- **dx_normal**: 정규화된 입력에 대한 gradient를 계산합니다.
$$dx_{normal} = dout \cdot \gamma$$
- **dlvar**: 분산에 대한 gradient를 계산합니다.
$$dlvar = \sum(dx_{normal} \cdot (x - \text{sample_mean}) \cdot -0.5 \cdot (\text{sample_var} + \text{eps})^{-1.5}, \text{axis}=0)$$
- **dlmean**: 평균에 대한 gradient를 계산합니다.
$$dlmean = \sum(dx_{normal} \cdot -1 / \sqrt{\text{sample_var} + \text{eps}}, \text{axis}=0) + dlvar \cdot \sum(-2 \cdot (x - \text{sample_mean}), \text{axis}=0) / N$$
- **dx**: 입력 데이터에 대한 gradient를 계산합니다.
$$dx = dx_{normal} \cdot 1 / \sqrt{\text{sample_var} + \text{eps}} + dlvar \cdot 2 \cdot (x - \text{sample_mean}) / N + dlmean / N$$

```
eps = 1e-5
N = dout.shape[0]
x, sample_mean, sample_var, gamma = cache
x_normal = (x - sample_mean) / np.sqrt(sample_var + eps)
dgamma = np.sum(dout * x_normal, axis = 0)
dbeta = np.sum(dout, axis = 0)
dx_normal = dout * gamma
dlvar = np.sum(dx_normal * (x - sample_mean) * -0.5 * (sample_var + eps)**-1.5, axis = 0)
dlmean = np.sum(dx_normal * -1 / np.sqrt(sample_var + eps), axis = 0)
+ dlvar * np.sum(-2 * (x - sample_mean), axis = 0) / N
dx = dx_normal * 1 / np.sqrt(sample_var + eps) + dlvar * 2 * (x - sample_mean) / N + dlmean / N
```

Dropout 구현

Train를 진행할 경우

```
mask = (np.random.random_sample(x.shape)<p)/p
out = mask * x
```

Test를 진행할 경우

```
if mode == "train":
    #####
    # TODO: Implement training phase backward pass for inverted dropout #
    #####
    # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
    dx = dout*mask

    # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
    #####
    #                                     END OF YOUR CODE                                     #
    #####
elif mode == "test":
    dx = dout
return dx
```

다음 주제: RNN의 이해와 CNN 구현